

Тема: Архитектура ПО

- 1. Понятие архитектуры ПО**
- 2. Архитектурная эволюция и деградация**
- 3. Архитектурные стили**

Что такое архитектура ПО?

- ☐ элементы (elements) - что (what)
- ☐ форма (form) - как (how)
- ☐ обоснование (rationale) - зачем (why)

Определение

Архитектура ПО включает ряд важных решений об организации программной системы, среди которых:

- **выбор структурных элементов и их интерфейсов, составляющих систему как единое целое**
- **поведение, обеспечиваемое совместной работой этих элементов**
- **организация этих элементов в более крупные подсистемы**
- **архитектурный стиль, которого придерживается данная организация**

Определение

Архитектура - ряд важных решений о системе.

Элементы проектирования архитектуры

- ☐ разделение системы на составные части в самом первом приближении
- ☐ принятие решений, которые будет сложно изменить впоследствии
- ☐ выработка множества различных вариантов организации системы
- ☐ может меняться в процессе жизненного цикла системы

Выбор архитектуры

- ☐ функциональность
- ☐ удобство использования
- ☐ устойчивость
- ☐ производительность
- ☐ повторное использование
- ☐ понятность
- ☐ экономические ограничения
- ☐ технологические ограничения
- ☐ эстетическое восприятие
- ☐ поиск компромиссов

Исходные вопросы проектирования

- ☐ Как пользователь будет использовать приложение?
- ☐ Как приложение будет развёртываться и обслуживаться при эксплуатации?
- ☐ Какие требования по атрибутам качества (безопасность, производительность, распараллеливание, интернационализация и т.д.)?
- ☐ Как спроектировать приложение, чтобы оно оставалось гибким и удобным в обслуживании в течении долгого времени?
- ☐ Основные архитектурные направления, которые могут оказывать влияние на приложение сейчас или после его развёртывания

Цели архитектуры

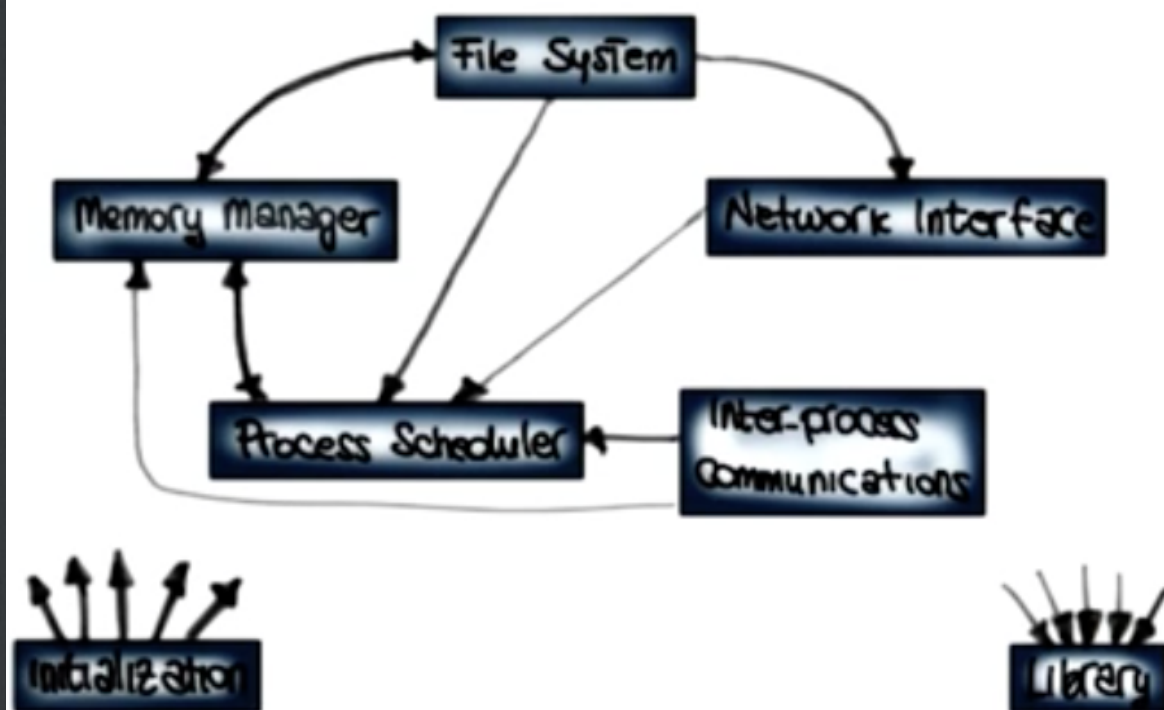
- ☐ раскрывать структуру системы, но скрывать детали реализации
- ☐ реализовывать все варианты использования и сценарии
- ☐ по возможности отвечать всем требованиям различных заинтересованных сторон
- ☐ выполнять требования как по функционалу, так и по качеству

Временный характер

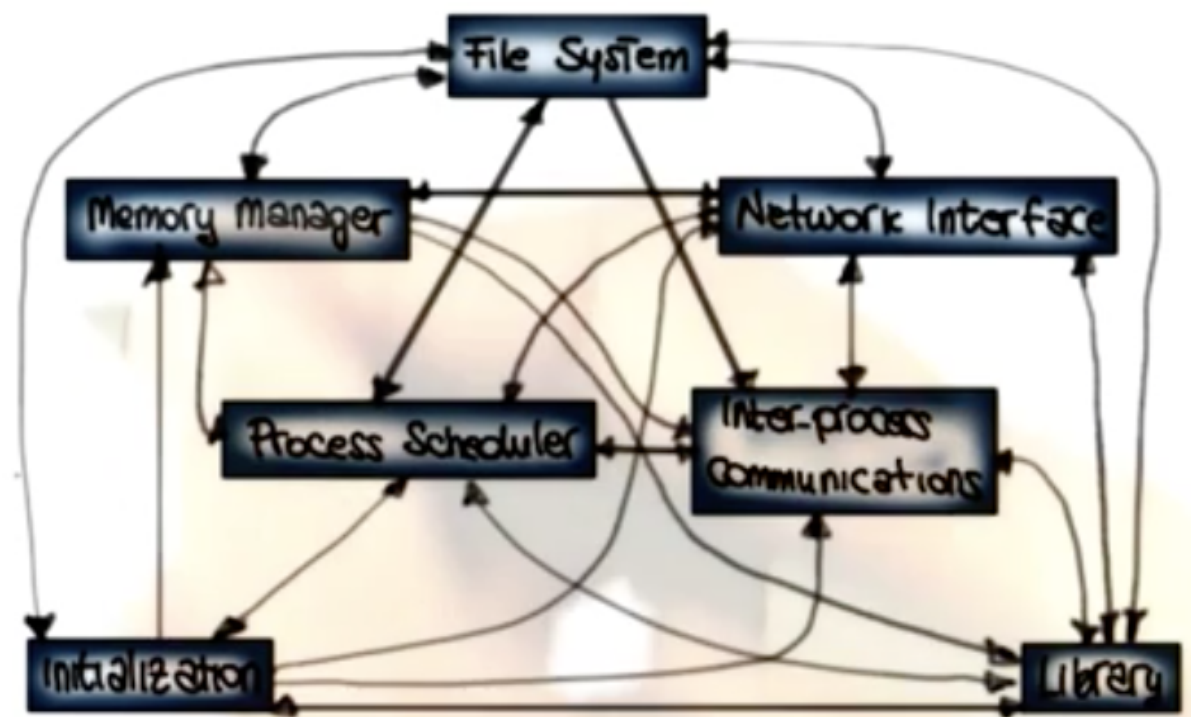
- ☐ предписывающая архитектура
- ☐ описывающая архитектура

Примеры

AN EXAMPLE FROM THE LINUX KERNEL



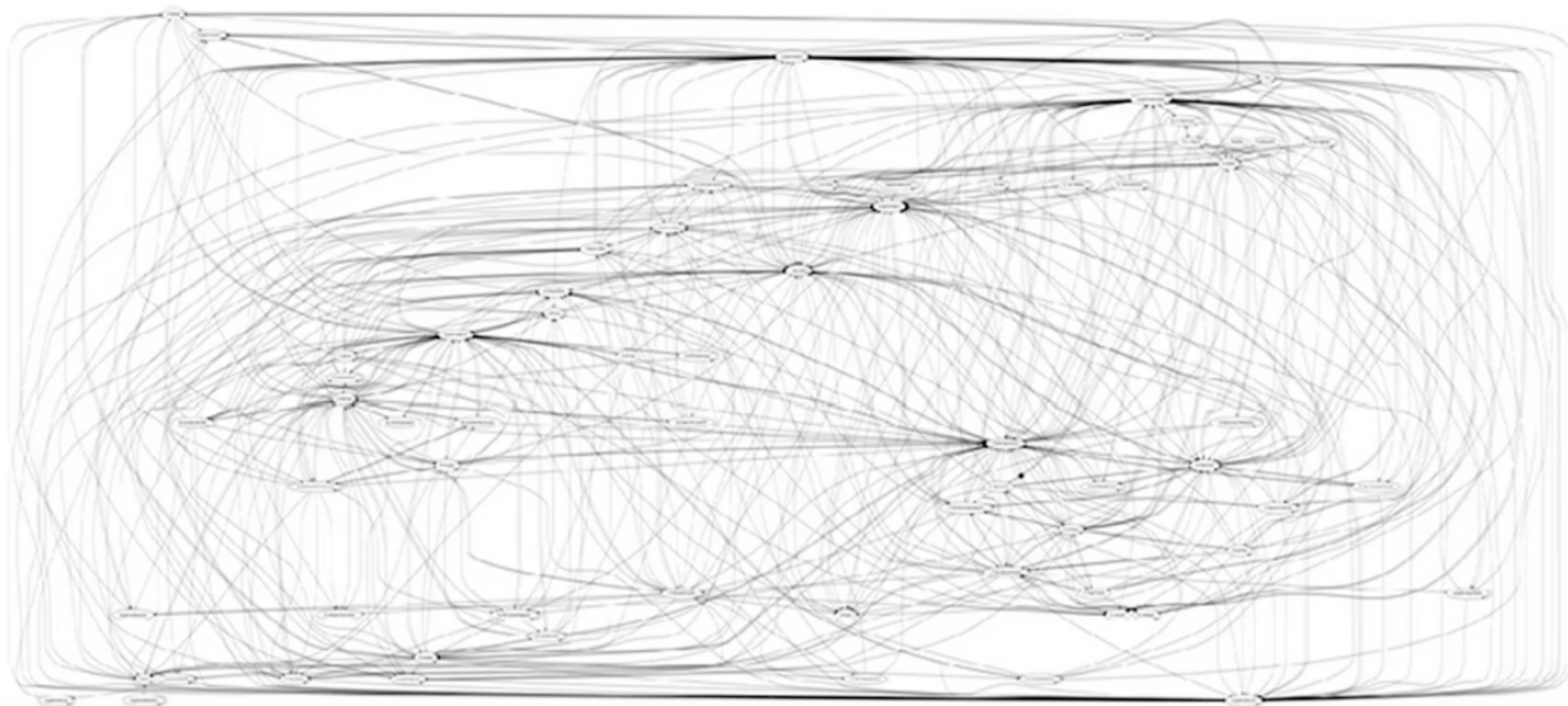
Prescriptive architecture



Descriptive architecture

Примеры

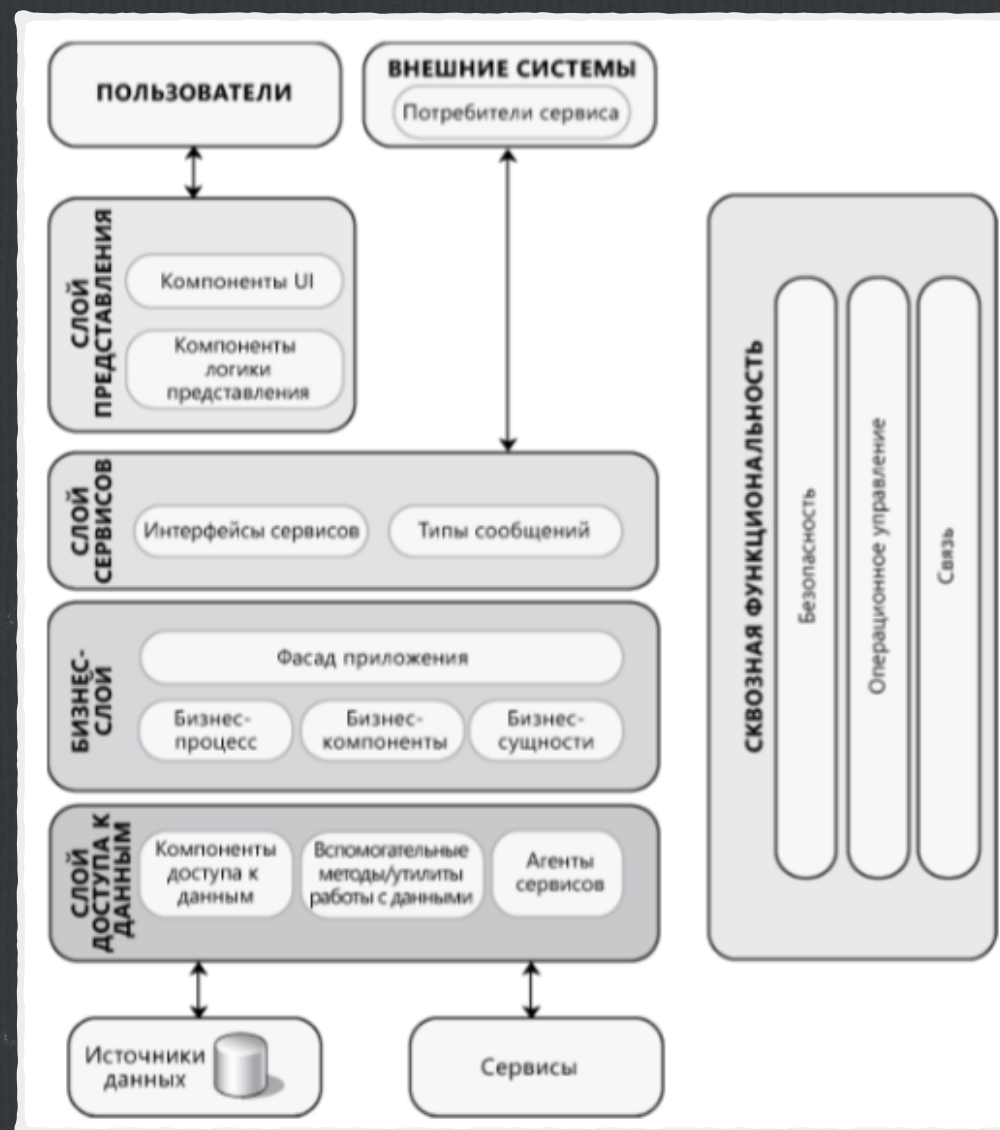
MORE EXAMPLES : HADOOP



Представление архитектуры системы

- ☐ **диаграммы компонентов**
- ☐ **диаграммы развёртывания**

Принципы проектирования архитектуры



- ☐ принцип разделения функций (high cohesion & low coupling)
- ☐ принцип единственной ответственности
- ☐ принцип минимального знания
- ☐ принцип DRY
- ☐ принцип минимизации проектирования (BDUF vs YAGNI)

Пример разбиения на функциональные области

- ☐ GUI
- ☐ выполнение бизнес процессов
- ☐ доступ к данным

Практики проектирования

- ☐ единообразие шаблонов проектирования в рамках одного слоя
- ☐ не дублировать функциональность
- ☐ композиция лучше наследования
- ☐ единый стиль кода и именования
- ☐ автоматизация QA
- ☐ условия эксплуатации приложения

Основные вопросы проектирования

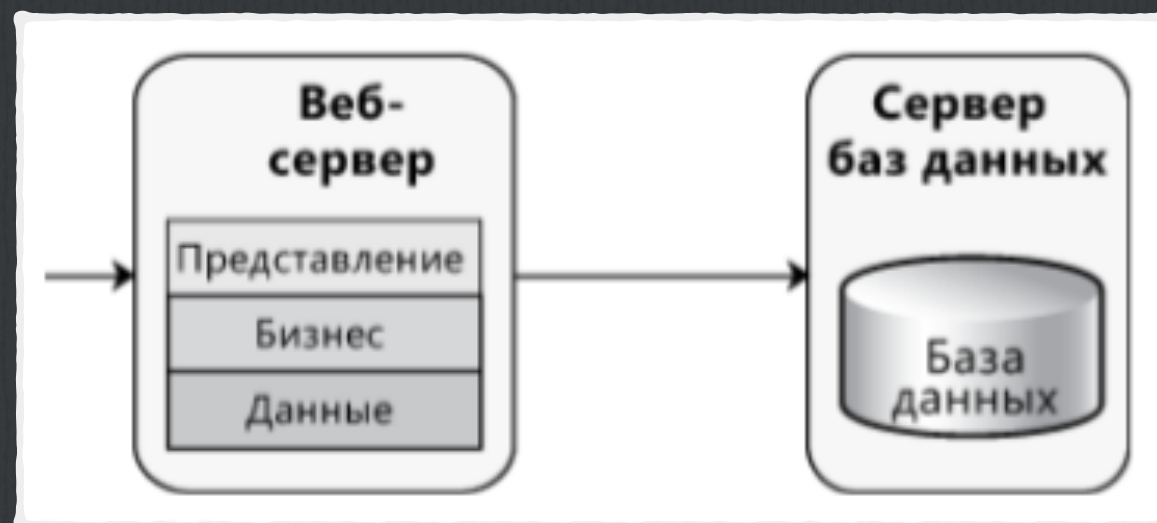
- ☐ определение типа приложения
- ☐ выбор стратегии развёртывания
- ☐ выбор соответствующих технологий
- ☐ выбор показателей качества
- ☐ реализация сквозной функциональности

Типы приложений

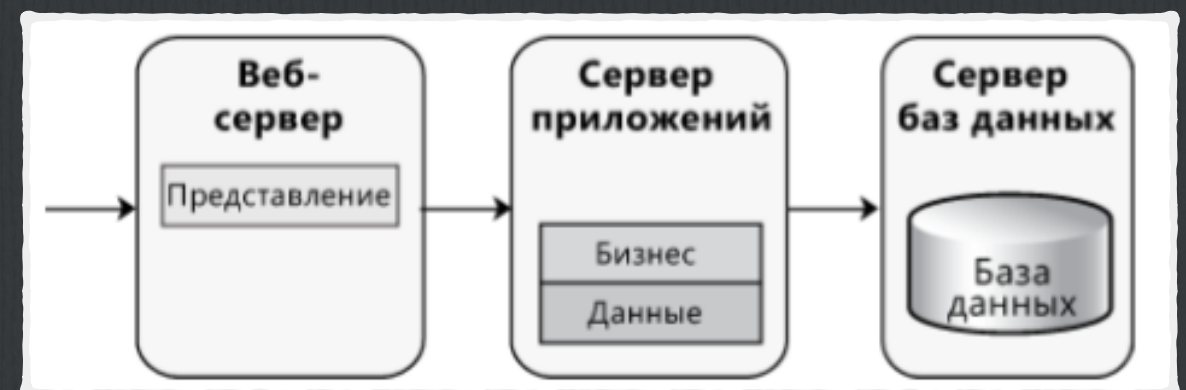
- ☐ приложения для мобильных устройств
- ☐ насыщенные клиентские приложения для выполнения на ПК
- ☐ насыщенные клиентские приложения для развёртывания из Интернета
- ☐ сервисы для обеспечения связи между слабо связанными компонентами
- ☐ веб-приложения для выполнения преимущественно на сервере

Стратегии развёртывания

Нераспределённое



Распределённое



с сохранением состояния и без

Выбор показателей качества

- ☐ **Каковы основные показатели качества приложения?**
- ☐ **Каковы основные требования к их реализации?**
- ☐ **Поддаются ли они количественному определению?**
- ☐ **Каковы критерии приёмки?**

Сквозная функциональность

- ☐ инструментирование и протоколирование
- ☐ аутентификация
- ☐ авторизация
- ☐ управление исключениями
- ☐ СВЯЗЬ
- ☐ кэширование

Архитектурные стили

- ☐ семейство систем с точки зрения схемы организации структуры
- ☐ определяет набор компонентов и соединений
- ☐ описывает ряд ограничений (например, топологических)

Архитектура клиент-сервер

Система разделяется на два приложения, где клиент выполняет запросы к серверу.

Во многих случаях в роли сервера выступает база данных, а логика приложения представлена процедурами хранения.

- ☐ безопасность
- ☐ централизованный доступ к данным
- ☐ простота обслуживания

Компонентная архитектура

Дизайн приложения разлагается на функциональные или логические компоненты с возможностью повторного использования, предоставляющие тщательно проработанные интерфейсы связи.

- ☐ повторное использование
- ☐ замещаемость
- ☐ независимость от контекста
- ☐ расширяемость
- ☐ инкапсуляция

Domain Driven Design

Объектно-ориентированный архитектурный стиль, ориентированный на моделирование сферы деловой активности и определяющий бизнес-объекты на основании сущностей этой сферы.

- ☐ обмен информацией
- ☐ расширяемость
- ☐ удобство тестирования

Многослойная архитектура

Функциональные области приложения разделяются на многослойные группы (уровни).

- ☐ абстракция
- ☐ инкапсуляция
- ☐ функциональные слои
- ☐ высокая связность
- ☐ повторное использование
- ☐ слабое связывание

Шина сообщений

Архитектурный стиль,
предписывающий
использование программной
системы, которая может
принимать и отправлять
сообщения по одному или
более каналам связи, так что
приложения получают
возможность
взаимодействовать, не
располагая конкретными
сведениями друг о друге.

- ☐ взаимодействие, основанное на сообщениях
- ☐ сложная логика обработки
- ☐ изменение логики обработки
- ☐ интеграция с разными инфраструктурами

Сервис-ориентированная архитектура

Описывает приложения, предоставляющие и потребляющие функциональность в виде сервисов с помощью контрактов и сообщений.

- ☐ сервисы автономны
- ☐ сервисы могут быть распределены
- ☐ сервисы слабо связаны
- ☐ сервисы совместно используют схему или контракт, но не класс
- ☐ совместимость основана на политике

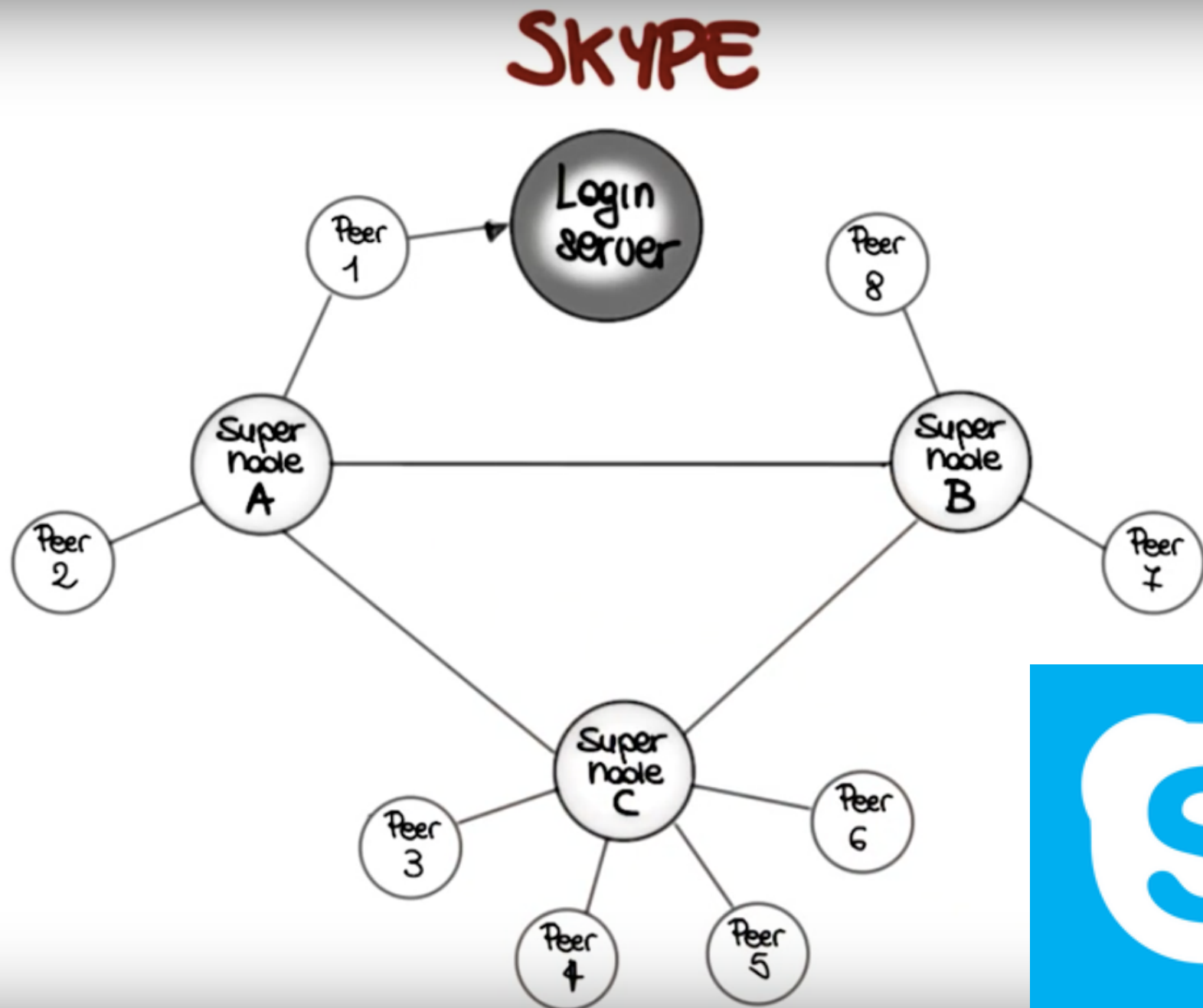
Ещё одна классификация

- ☐ pipes & filters
- ☐ event driven architecture
- ☐ client-server
- ☐ publisher-subscriber
- ☐ p2p
- ☐ REST

Выбор стиля

- ☐ требования бизнеса
- ☐ требования качества
- ☐ особенности проекта
- ☐ сочетания стилей

Пример



Гибкое моделирование архитектуры

4th Edition



АРХИТЕКТУРА

КАКАЯ ЕЩЕ АРХИТЕКТУРА?

*все о трех буквах на которые вас послал
проджект менеджер*

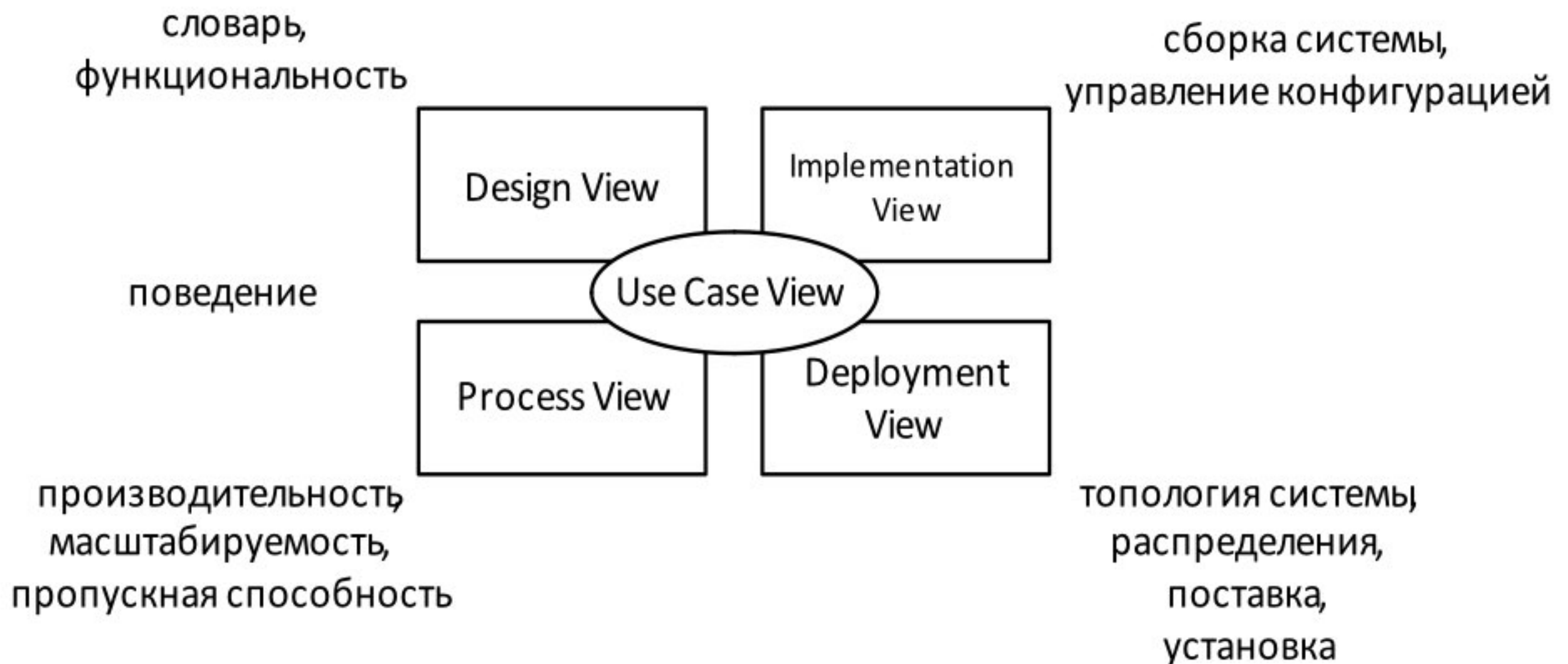
O'REILLY®

Василий Пупкин

Представление архитектуры системы (4 + 1)

- ☐ **Логическое представление**
- ☐ **Представление процессов**
- ☐ **Представление разработки**
- ☐ **Физическое представление**
- ☐ **Представление сценариев**

Архитектура программной системы



Процесс моделирования

- ☐ сеансы гибкого моделирования
- ☐ применение стандартов моделирования
- ☐ простота, эволюционность, развитие команды

Архитектура ПО: Слои



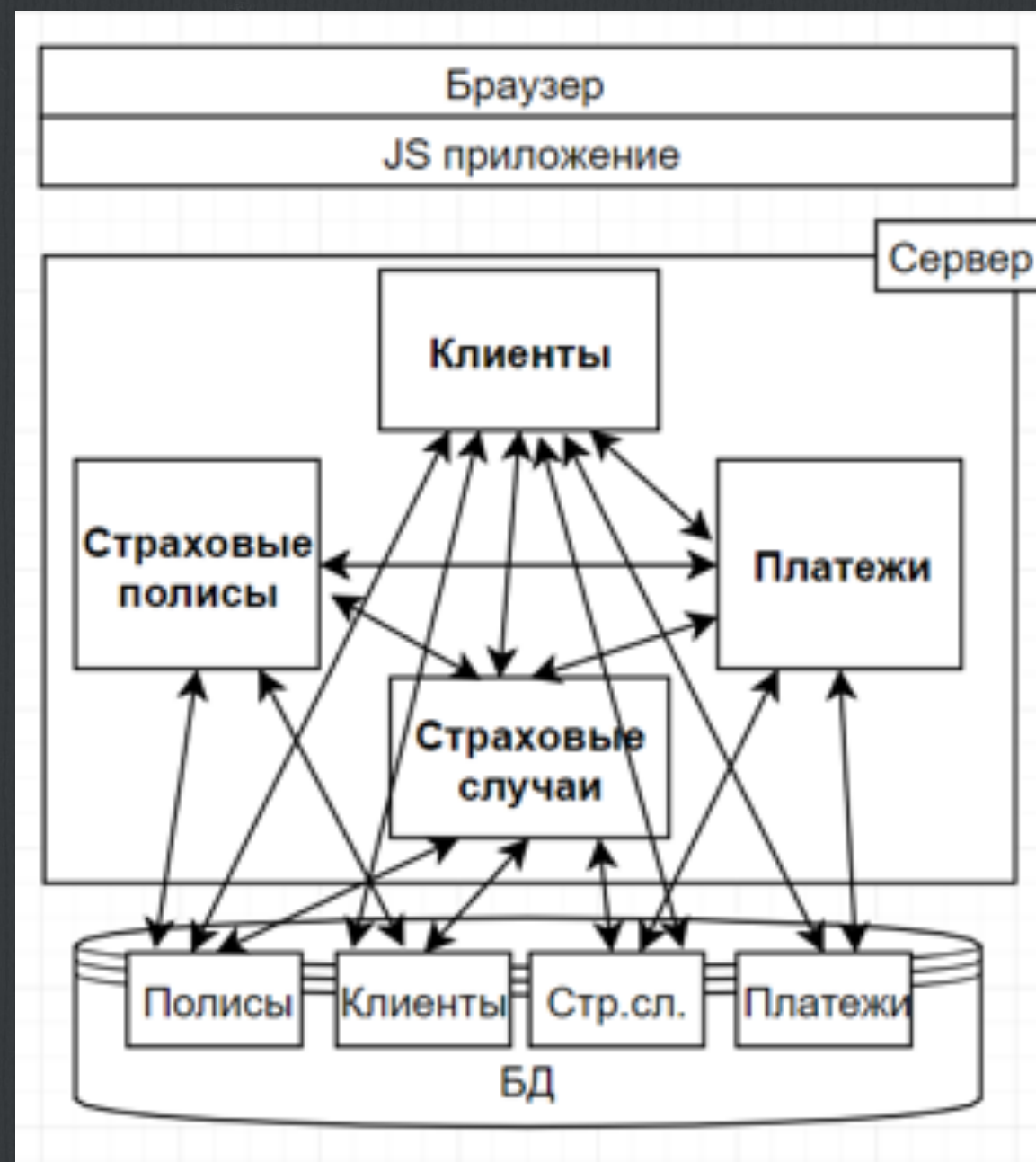
Архитектура ПО: Компоненты



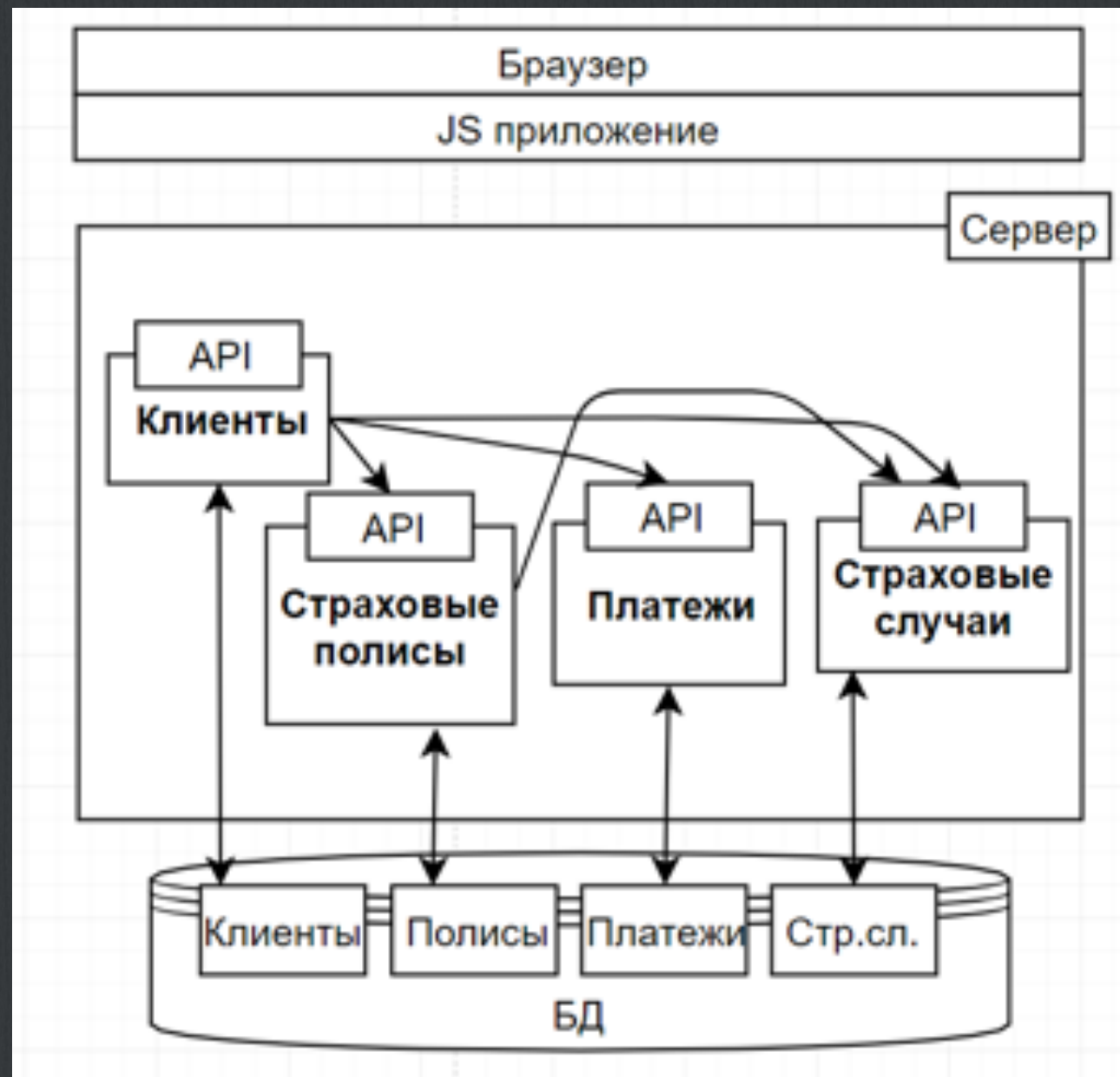
Архитектура ПО: Модули



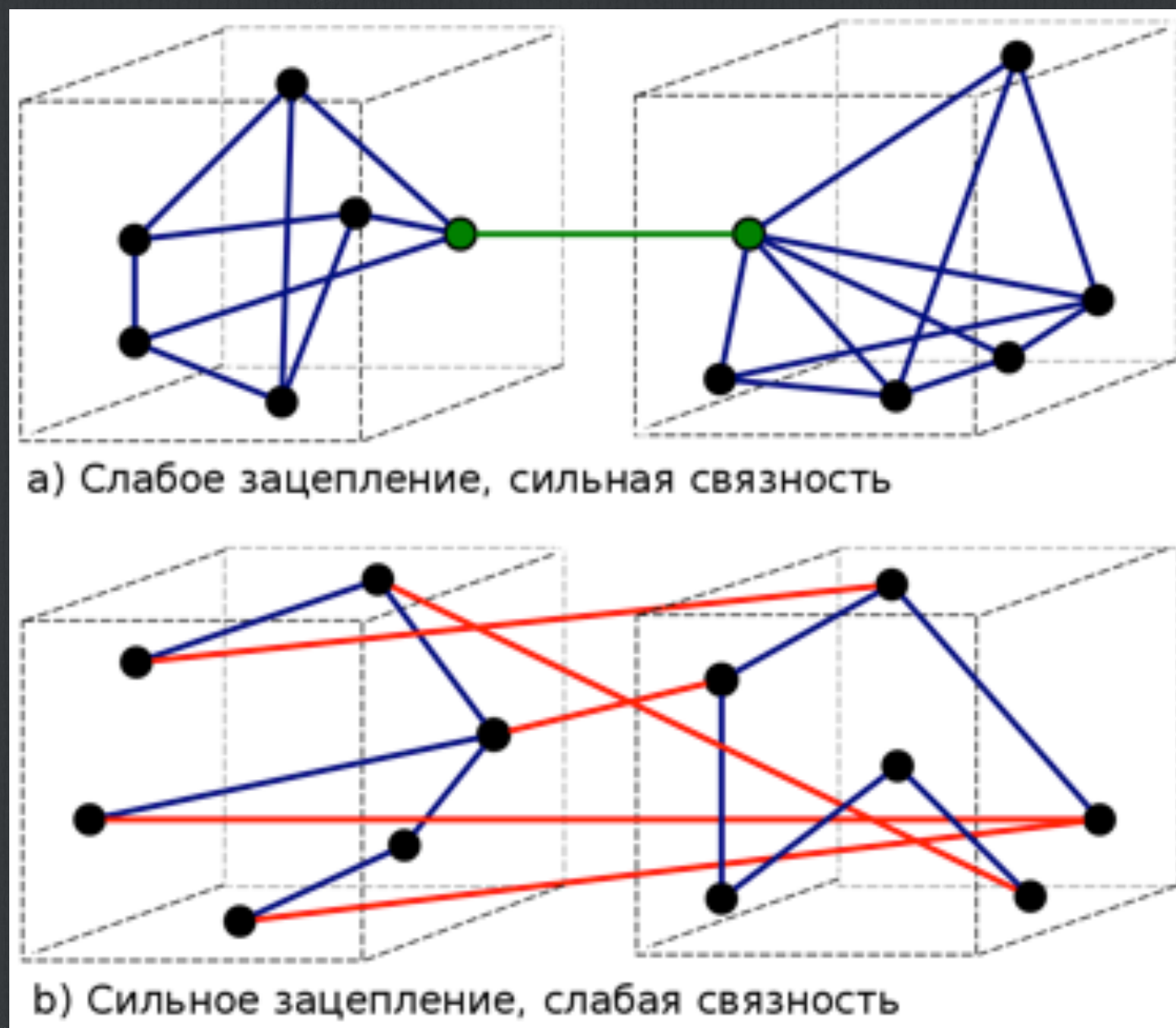
Виды архитектур: VVoM



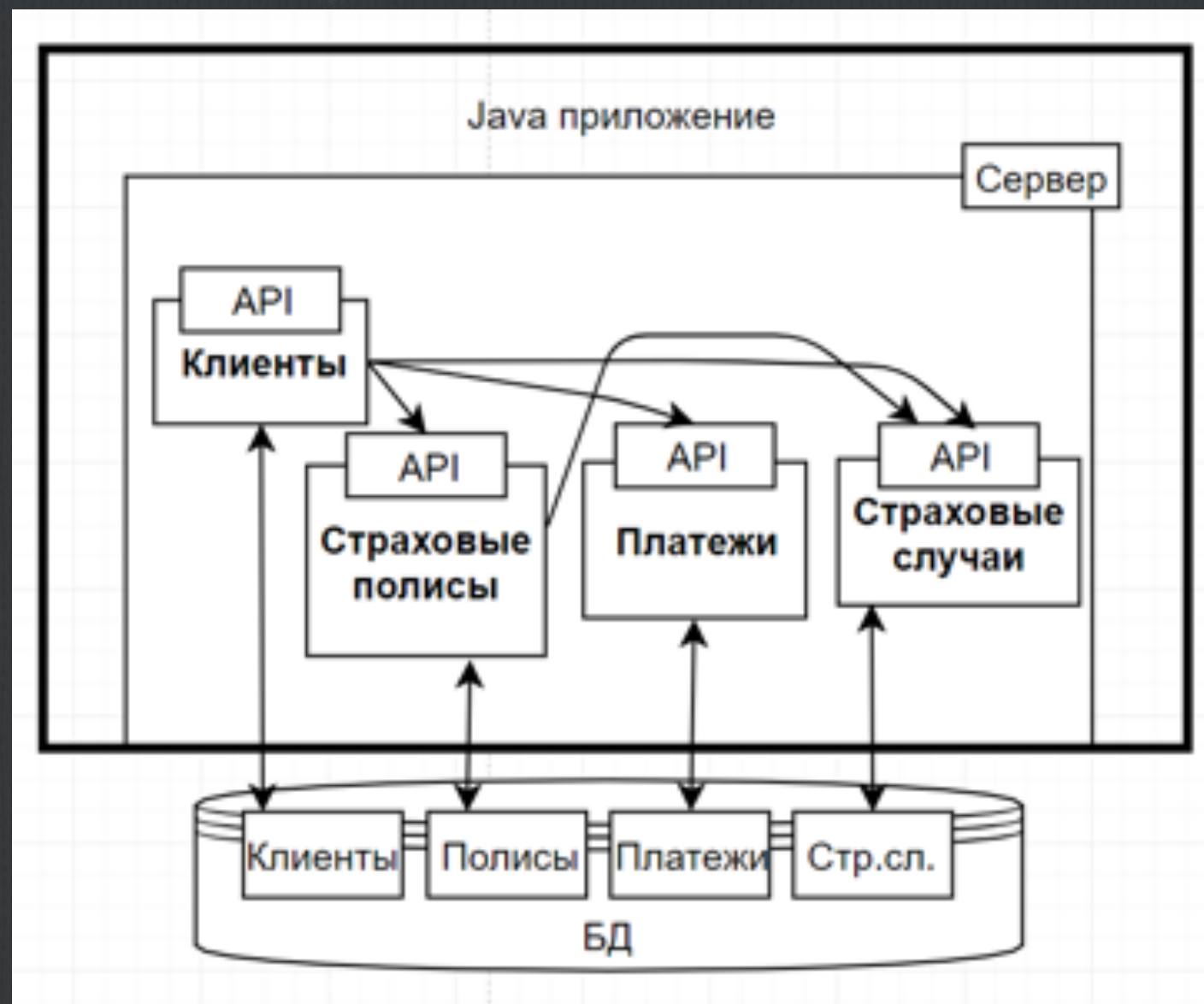
Виды архитектур: SOA



Виды архитектур: SOA



SOA: Монолит



SOA: Микросервисы

